

# CHAPTER 12

## 레이블링과 외곽선 검출

OpenCV 4로 배우는 컴퓨터 비전과 머신 러닝

# 12장 레이블링과 외곽선 검출

---

## 12.1 레이블링

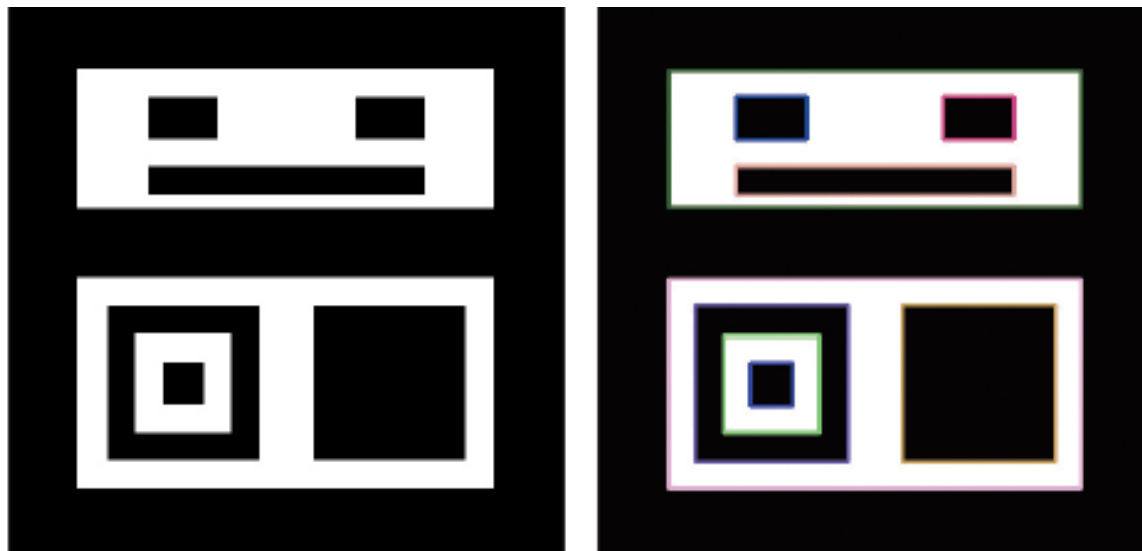
## 12.2 외곽선 검출

## 12.2 외곽선 검출

---

# 외곽선 검출

- 객체의 외곽선(contour)은 객체 영역 픽셀 중에서 배경 영역과 인접한 픽셀들의 집합
- 보통 검은색 배경 안에 있는 흰색 객체 영역에서 가장 최외곽에 있는 픽셀을 찾아 외곽선으로 정의함
- 만약 흰색 객체 영역 안에 검은색 배경 영역인 홀(hole)이 존재한다면 홀을 둘러싸고 있는 객체 픽셀들도 외곽선으로 검출 -> 객체의 외곽선 = 객체 바깥쪽 외곽선 + 안쪽 홀 외곽선



# 외곽선 검출

- 객체 하나의 외곽선은 여러 개의 점으로 구성됨 -> 객체 하나의 외곽선 정보는 `vector<Point>` 타입의 객체에 저장
- 하나의 영상에는 여러 개의 객체가 존재할 수 있으므로 영상 하나에서 추출된 전체 객체의 외곽선 정보는 `vector<vector<Point>>` 타입의 객체에 저장함
- OpenCV에서는 다음과 같은 형식의 변수를 선언하여 외곽선 검출 함수에 전달함

```
vector<vector<Point>> contours;
```

|             |                                  |
|-------------|----------------------------------|
| contours[0] | <code>vector&lt;Point&gt;</code> |
| contours[1] | <code>vector&lt;Point&gt;</code> |
| ⋮           | ⋮                                |
| contours[n] | <code>vector&lt;Point&gt;</code> |

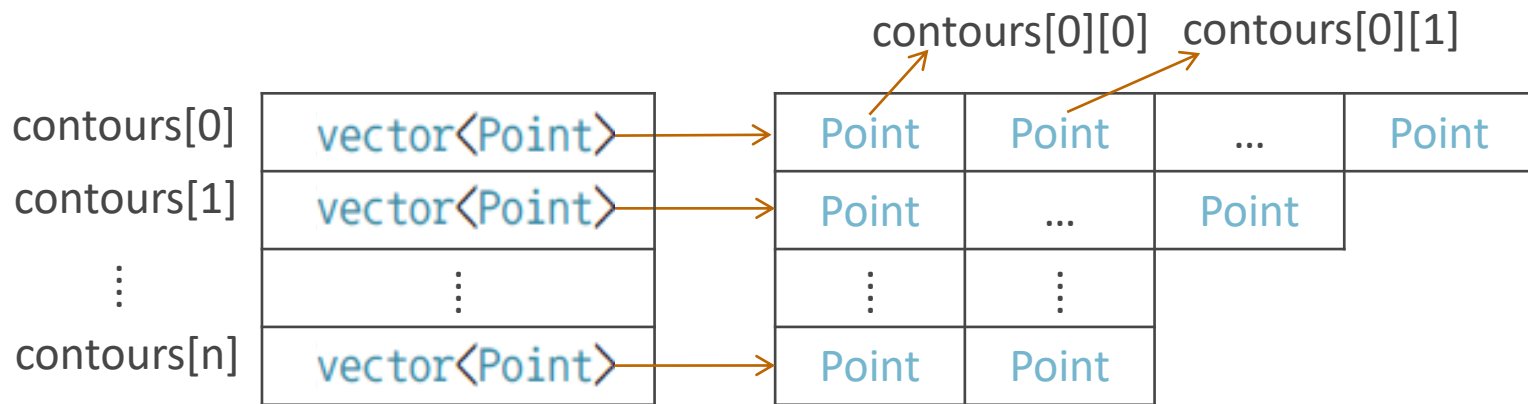
원소의갯수 : `contours.size()`

**i번째 외곽선 : `contours[i]`**

# 외곽선 검출

- 외곽선을 구성하는 점에 접근하는 방법

```
vector<vector<Point>> contours;
```



외곽선의 갯수 : contours.size()

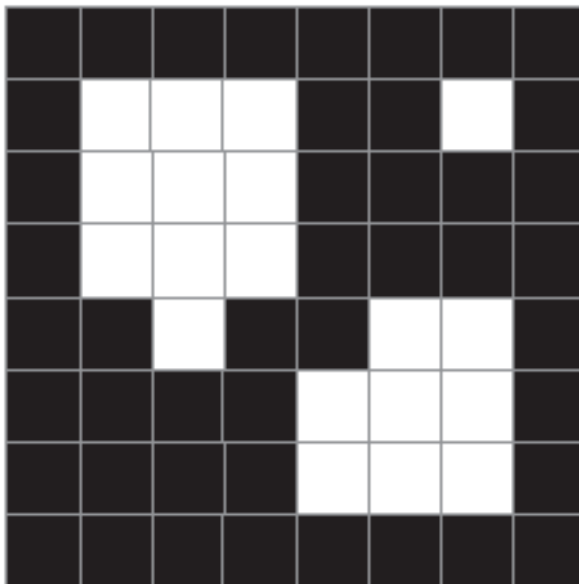
i번째 외곽선의 크기 : contours[i].size()

i번째 외곽선의 j번째 점 : contours[i][j]

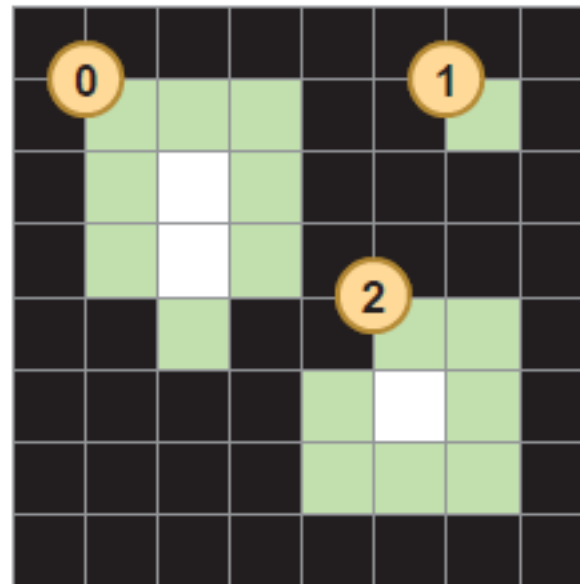
# 외곽선 검출

- 그림 12-6(a) : 8×8 크기의 이진 영상, 흰색으로 표현된 3개의 객체 영역과 홀은 없는 단순한 형태임
- 이 영상에 대해 외곽선 검출을 수행하면 그림 12-6(b)에서 녹색으로 표현한 픽셀들이 외곽선 점들로 검출됨

▼ 그림 12-6 외곽선 검출 결과 분석을 위한 테스트 영상



(a)



(b)

# 외곽선 검출

- contours 변수는 `vector<vector<Point>>` 타입이며, 이는 `vector<Point>` 타입을 여러 개 포함할 수 있는 타입임
- 그림 12-6에서는 외곽선 객체가 3개 있으므로 contours 변수에는 `vector<Point>` 타입의 데이터가 3개 저장됨
- contours 변수에 저장된 각각의 외곽선은 `contours[0]`, `contours[1]`, `contours[2]` 형태로 접근할 수 있고 모두 `vector<Point>` 타입임
- 각각의 외곽선 `contours[i]`는 i번째 외곽선 점들의 좌표를 저장하고 있음(좌측상단부터 반시계방향으로 저장됨)

`contours[0]` : [1, 1], [1, 2], [1, 3], [2, 4], [3, 3], [3, 2], [3, 1], [2, 1]

`contours[1]` : [6, 1]

`contours[2]` : [5, 4], [4, 5], [4, 6], [5, 6], [6, 6], [6, 5], [6, 4]

- 외곽선의 개수 : `contours.size()`
- 0번째 외곽선의 점들의 개수 : `contours[0].size()`
- 0번째 외곽선의 3번째 점의 좌표 : `contours[0][3] -> (2, 4)`

# 외곽선 검출

```
void findContours(InputArray image, OutputArrayOfArrays contours,
                 OutputArray hierarchy, int mode,
                 int method, Point offset = Point());
void findContours(InputArray image, OutputArrayOfArrays contours,
                 int mode, int method, Point offset = Point());
```

- **image**      입력 영상. 8비트 1채널 영상이어야 하고, 0이 아닌 픽셀을 객체로 취급합니다. 만약 mode가 RETR\_CCOMP이면 CV\_32SC1 타입의 영상을 지정할 수 있습니다.
- **contours**    검출된 외곽선 정보. vector<vector<Point>> 타입의 변수를 지정합니다.
- **hierarchy**    외곽선 계층 정보. vector<Vec4i> 타입의 변수를 지정합니다.
- **mode**        외곽선 검출 모드. RetrievalModes 열거형 상수를 지정합니다.
- **method**      외곽선 근사화 방법. ContourApproximationModes 열거형 상수를 지정합니다.
- **offset**      외곽선 점 좌표의 오프셋(이동 변위)

- findContours()함수는 외곽선들의 계층 정보를 받아 오는 hierarchy 인자가 있는 형태와 없는 형태 두 가지로 정의되어 있음

# 외곽선 검출

- mode : 외곽선을 어떤 방식으로 검출할 것인지를 나타내는 검출 모드를 지정하고 RetrievalModes 열거형 상수 중 하나를 지정

## ▼ 표 12-1 주요 RetrievalModes 열거형 상수

| RetrievalModes 열거형 상수 | 설명                                         |
|-----------------------|--------------------------------------------|
| RETR_EXTERNAL         | 객체 바깥쪽 외곽선만 검색합니다. 계층 구조는 만들지 않습니다.        |
| RETR_LIST             | 객체 바깥쪽과 안쪽 외곽선을 모두 검색합니다. 계층 구조는 만들지 않습니다. |
| RETR_CCOMP            | 모든 외곽선을 검색하고 2단계 계층 구조를 구성합니다.             |
| RETR_TREE             | 모든 외곽선을 검색하고 전체 계층 구조를 구성합니다.              |

# 외곽선 검출

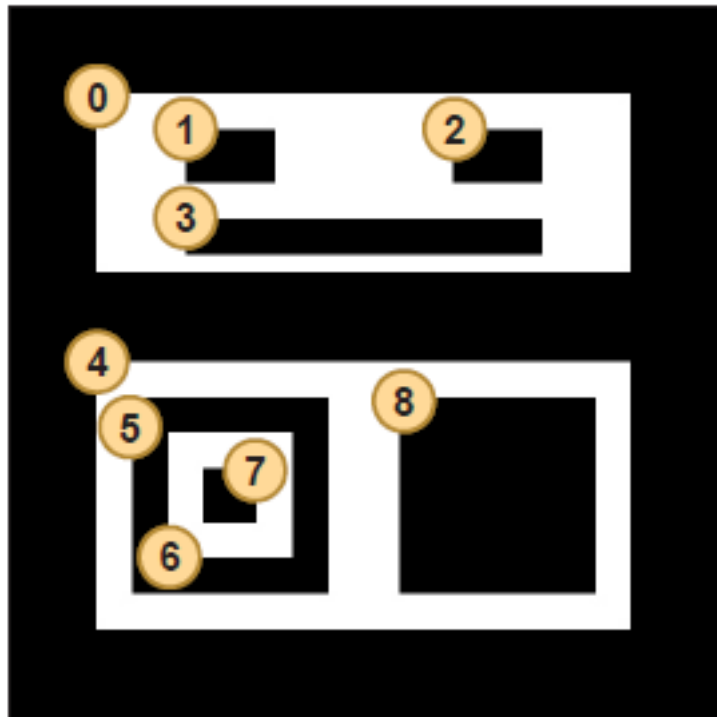
- method : 검출된 외곽선 점들의 좌표를 근사화하는 방법을 지정하고 ContourApproximationModes 열거형 상수 중에 하나를 지정
- 저장되는 외곽선 점의 개수를 줄이고 싶다면 CHAIN\_APPROX\_SIMPLE 상수를 사용하면 유리함
- CHAIN\_APPROX\_TC89\_L1 또는 CHAIN\_APPROX\_TC89\_KCOS 방식은 점의 개수는 많이 줄어들지만 외곽선 모양에 변화가 생기므로 주의해야 함

▼ 표 12-2 ContourApproximationModes 열거형 상수

| ContourApproximationModes 열거형 상수 | 설명                                   |
|----------------------------------|--------------------------------------|
| CHAIN_APPROX_NONE                | 모든 외곽선 점들의 좌표를 저장합니다.                |
| CHAIN_APPROX_SIMPLE              | 외곽선 중에서 수평선, 수직선, 대각선 성분은 끝점만 저장합니다. |
| CHAIN_APPROX_TC89_L1             | Teh & Chin L1 근사화를 적용합니다.            |
| CHAIN_APPROX_TC89_KCOS           | Teh & Chin k cos 근사화를 적용합니다.         |

# 외곽선 검출

- 간단한 테스트 영상 : 검은색 배경에 큰 흰색 객체가 두 개 있고, 각각의 객체 안에는 검은색 홀이 여러 개 있음, 홀 안쪽에는 다시 작은 흰색 객체가 있음
- 외곽선의 계층 구조는 외곽선의 포함 관계에 의해 결정됨



- 0 번 외곽선 안에는 1·2·3 번 홀 외곽선이 있으므로 0 번 외곽선은 1·2·3 번 외곽선의 부모 외곽선이고, 1·2·3 번 외곽선은 0 번 외곽선의 자식 외곽선임
- 0 번과 4 번 외곽선은 서로 포함 관계가 없이 대등하므로 이전 외곽선 또는 다음 외곽선의 관계를 가짐

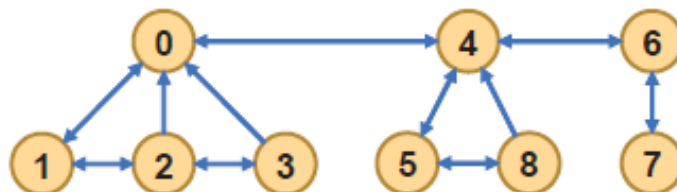
▼ 그림 12-7 외곽선 계층 구조 확인을 위한 테스트 영상

# 외곽선 검출

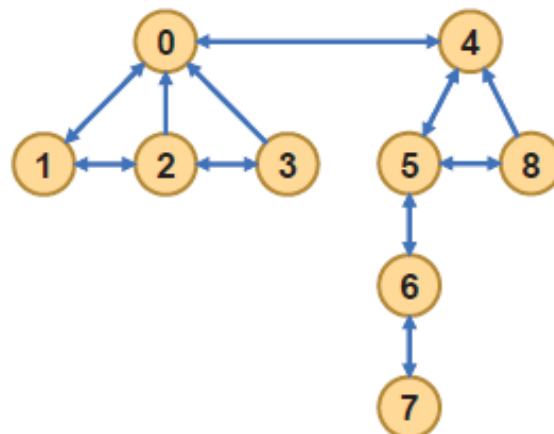
## ▼ 그림 12-8 외곽선 검출 모드에 따른 계층 구조



RETR\_CCOMP



RETR\_TREE



# 외곽선 검출

- 그림 12-8 : 4가지 외곽선 검출 모드에 따른 검출 결과와 계층 구조
  - 각 외곽선 번호 사이에 연결된 화살표는 계층 구조를 나타냄
  - 화살표가 오른쪽 외곽선 번호를 가리키면 다음 외곽선을 나타냄
  - 화살표가 왼쪽 외곽선 번호를 가리키면 이전 외곽선을 나타냄
  - 화살표가 아래쪽을 가리키면 자식 외곽선을 나타냄
  - 화살표가 위쪽을 가리키면 부모 외곽선을 나타냄
- RETR\_EXTERNAL 검출 모드
  - 흰색 객체의 바깥쪽 외곽선만 검출함
  - 이 경우 객체 내부의 홀 외곽선은 검출되지 않음, 큰 객체 내부에 있는 작은 객체의 외곽선도 검출되지 않음
  - 그림 12-7의 테스트 영상에서 RETR\_EXTERNAL 모드로 외곽선을 검출하면 0번과 4번 외곽선만 검출됨

# 외곽선 검출

- RETR\_LIST 검출 모드

- 바깥쪽과 안쪽 홀 외곽선을 모두 검출함
- RETR\_EXTERNAL 또는 RETR\_LIST 모드를 사용할 경우, 외곽선의 부모/자식 계층 정보는 생성되지 않음

- RETR\_CCOMP 검출 모드

- 모든 흰색 객체의 바깥쪽 외곽선을 먼저 검출함
- 각 객체 안의 홀 외곽선을 자식 외곽선으로 설정함
- 상하 계층이 최대 두 개 층으로만 구성됨
- 만약 흰색 객체에 여러 개의 홀이 존재할 경우, 그 중 하나만 자식 외곽선으로 설정됨
- 각각의 홀 외곽선은 객체 바깥쪽 외곽선을 모두 부모 외곽선으로 설정함

- RETR\_TREE 검출 모드

- 외곽선 전체의 계층 구조를 생성함
- 만약 객체 내부에 홀이 있고, 그 홀 안에 또 다른 작은 객체가 있다면 작은 객체의 외곽선은 홀 외곽선의 자식으로 설정됨

## 예제 : 외곽선 검출

```
#include <iostream>
using namespace std;
#include <opencv2/opencv.hpp>
using namespace cv;
int main(void)
{
    uchar data[] = {
        0, 0, 1, 1, 0, 0, 0, 0,
        1, 1, 1, 1, 0, 0, 0, 0,
        1, 1, 1, 1, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 1, 1, 0,
        0, 0, 0, 1, 1, 1, 1, 0,
        0, 0, 0, 1, 1, 1, 1, 0,
        0, 0, 1, 1, 1, 1, 1, 0,
        0, 0, 0, 0, 0, 0, 0, 0,
    };

    Mat src = Mat(8, 8, CV_8UC1, data) * 255;
```

## 예제 : 외곽선 검출

```
vector<vector<Point>> contours;
findContours(src, contours, RETR_LIST, CHAIN_APPROX_NONE);
cout << "외곽선의 갯수:" << contours.size() << endl;
for(int i=0; i< contours.size(); i++)
{
    cout << i << "번째 외곽선의 점의 갯수:" << contours[i].size() << endl;
    cout << i << "번째 외곽선의 점의 좌표:" << endl;
    for (int j = 0; j < contours[i].size(); j++) cout << contours[i][j] << ',';
    cout << endl;
    cout << i << "번째 외곽선의 첫번째 점의 좌표:" << contours[i][0]
        << endl;
    cout << i << "번째 외곽선의 마지막 점의 좌표:"
        << contours[i][contours[i].size()-1] << endl;
    cout << endl;
}
return 0;
}
```

# 예제 : 외곽선 검출

Microsoft Visual Studio 디버그 콘솔

외곽선의 갯수:2

0번째 외곽선의 점의 갯수:12

0번째 외곽선의 점의 좌표:

[5, 3], [4, 4], [3, 4], [3, 5], [2, 6], [3, 6], [4, 6], [5, 6], [6, 6], [6, 5], [6, 4], [6, 3],

0번째 외곽선의 첫번째 점의 좌표: [5, 3]

0번째 외곽선의 마지막 점의 좌표: [6, 3]

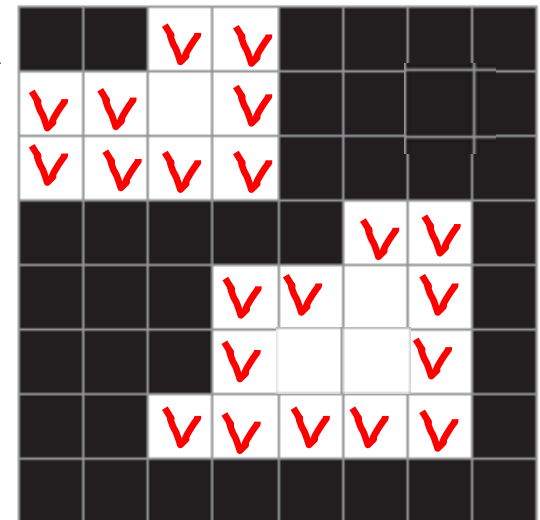
1번째 외곽선의 점의 갯수:9

1번째 외곽선의 점의 좌표:

[2, 0], [1, 1], [0, 1], [0, 2], [1, 2], [2, 2], [3, 2], [3, 1], [3, 0],

1번째 외곽선의 첫번째 점의 좌표: [2, 0]

1번째 외곽선의 마지막 점의 좌표: [3, 0]



# 외곽선 그리기

```
void drawContours(InputOutputArray image, InputArrayOfArrays contours,
                 int contourIdx, const Scalar& color,
                 int thickness = 1, int lineType = LINE_8,
                 InputArray hierarchy = noArray(),
                 int maxLevel = INT_MAX, Point offset = Point());
```

- **image** 외곽선을 그릴 영상
- **contours** findContours() 함수로 구한 전체 외곽선 정보. vector<vector<Point>> 타입의 변수를 지정합니다.
- **contourIdx** 외곽선 번호. 음수를 지정하면 전체 외곽선을 그립니다.
- **color** 외곽선 색상(또는 밝기)
- **thickness** 외곽선 두께. FILLED 또는 -1을 지정하면 외곽선 내부를 채웁니다.
- **lineType** 외곽선 타입
- **hierarchy** 외곽선 계층 정보
- **maxLevel** 그릴 외곽선의 최대 레벨. 이 값이 0이면 지정한 번호의 외곽선만 그리고, 1보다 같거나 크면 그에 해당하는 하위 레벨의 외곽선까지 그립니다.
- **offset** 추가적으로 지정할 외곽선 점 좌표의 오프셋(이동 범위). 지정한 좌표 크기만큼 외곽선 좌표를 이동하여 그립니다.

# 외곽선 그리기

- `findContours()` 함수로 검출한 외곽선 정보를 이용하여 영상 위에 외곽선을 그리고 싶다면 `drawContours()` 함수를 사용할 수 있음
- `drawContours()` 함수는 `findContours()` 함수로 얻은 외곽선 정보를 이용하여 영상에 외곽선을 그림
- 전체 외곽선을 한꺼번에 그릴 수도 있고, 특정 번호의 외곽선을 선택하여 그릴 수도 있음
- 외곽선 계층 정보를 함께 지정할 경우, 자식 외곽선도 함께 그릴 수 있음

## 코드12-3 : 외곽선 검출과 그리기

- 코드 12-3는 이진 입력 영상에서 모든 외곽선을 찾아 각기 다른 색상으로 외곽선을 그림

## 코드12-3 : 외곽선 검출과 그리기

```
#include <iostream>
using namespace std;
#include "opencv2/opencv.hpp"
using namespace cv;

int main(void)
{
    Mat src = imread("contours.bmp", IMREAD_GRAYSCALE);
    if (src.empty()) { cerr << "Image load failed!" << endl; return -1;}

    vector<vector<Point>> contours;
    findContours(src, contours, RETR_LIST, CHAIN_APPROX_NONE);

    Mat dst;
    cvtColor(src, dst, COLOR_GRAY2BGR);
```

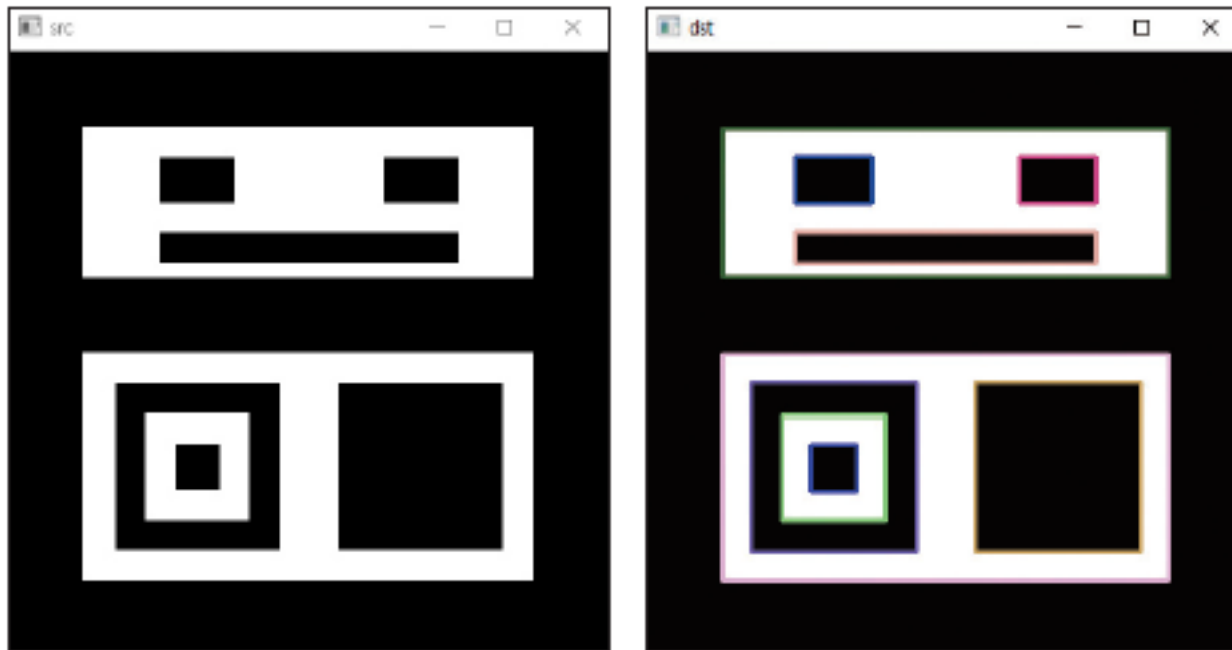
## 코드12-3 : 외곽선 검출과 그리기

```
for (int i = 0; i < contours.size(); i++) {  
    Scalar c(rand() & 255, rand() & 255, rand() & 255);  
    drawContours(dst, contours, i, c, 2);  
}  
  
imshow("src", src);  
imshow("dst", dst);  
waitKey(0);  
return 0;  
}
```

- rand() : 0에서 **RAND\_MAX**(32767) 범위의 랜덤 정수를 반환 합니다. stdlib.h 포함해야 함
- rand() & 255 : 하위 8비트를 제외한 상위비트는 0으로 만듦 -> 0~255범위의 정수로 만들어줌

## 코드12-3 : 외곽선 검출과 그리기

- 검출된 전체 외곽선 개수는 `contours.size()` 코드를 이용하여 알 수 있음, 사이즈는 9임
- 코드 12-3에서는 for 문을 0부터 `contours.size()`까지 반복하며 외곽선을 그림



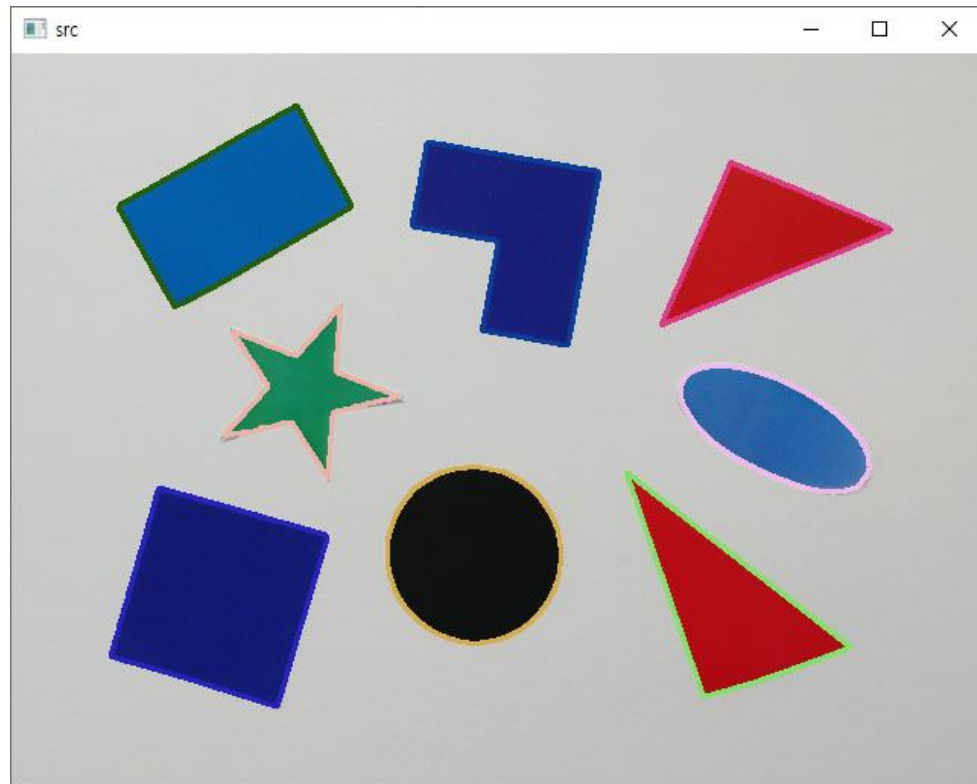
▼ 그림 12-9 계층 구조를 사용한 외곽선 검출과 그리기 예제 실행 결과

## 코드12-4 : 계층구조를 사용한 외곽선 검출

- 생략

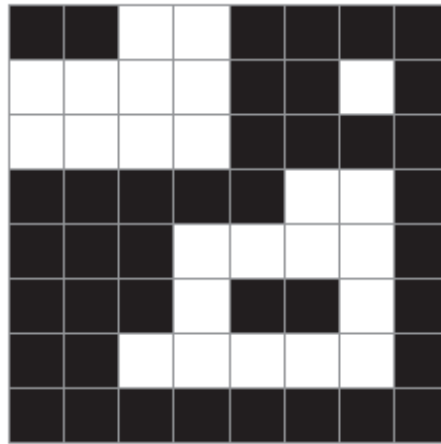
# 실습과제1

- polygon.bmp 영상파일에서 아래 처럼 객체의 외곽선을 그려주는 프로그램을 작성하시오.



## 실습과제2

- 코드12-1의 Mat 객체에 저장된 영상에 대하여 모든 외곽선을 찾아서 각 외곽선을 구성하는 점들을 모두 화면에 출력해 보시오.

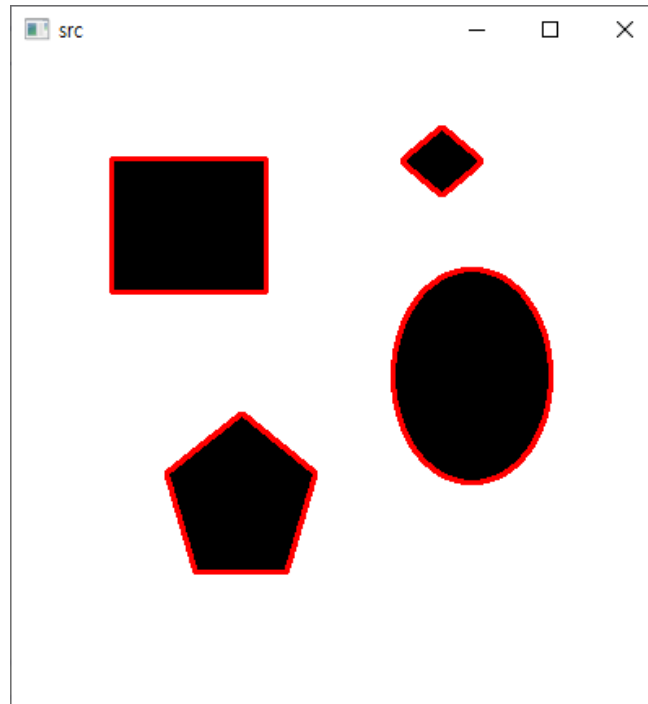


```

Microsoft Visual Studio 디버그 콘솔
외곽선의 갯수:4
0번째 외곽선의 점의 좌표:
[3, 5],[4, 4],[5, 4],[6, 5],[5, 6],[4, 6],
1번째 외곽선의 점의 좌표:
[5, 3],[4, 4],[3, 4],[3, 5],[2, 6],[3, 6],[4, 6],[5, 6],[6, 6],[6, 5],[6, 4],[6, 3],
2번째 외곽선의 점의 좌표:
[6, 1],
3번째 외곽선의 점의 좌표:
[2, 0],[1, 1],[0, 1],[0, 2],[1, 2],[2, 2],[3, 2],[3, 1],[3, 0],
  
```

## 실습과제3

- labelex.png 영상에서 drawContours() 함수를 사용하지 말고 외곽선을 직접 그려주는 코드를 작성하시오.
- 외곽선을 구성하는 모든 이웃하는 2개의 점을 직선(line함수)으로 이어주면 됨

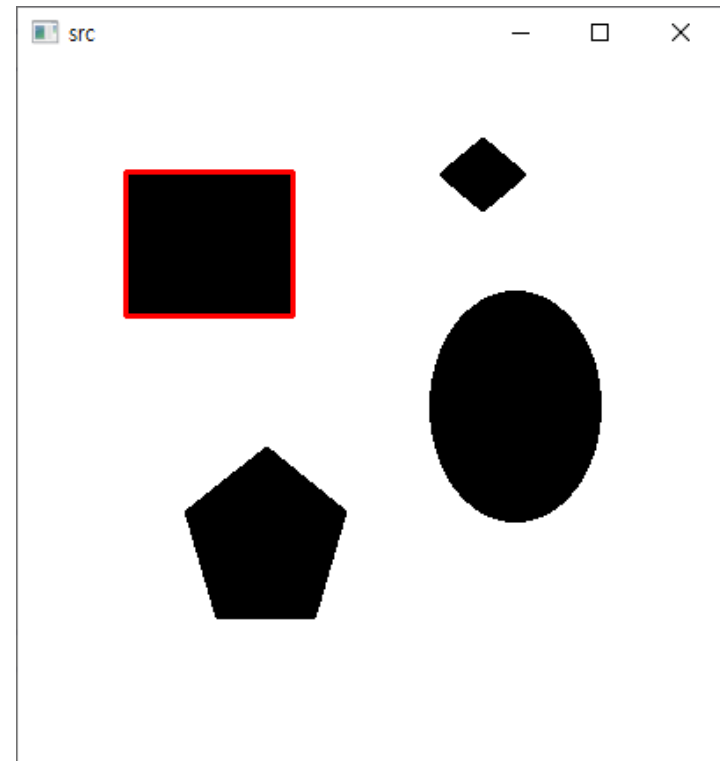


## 실습과제4

- labelex.png 영상에서 외곽선의 길이를 출력하고 외곽선의 길이가 최대인 외곽선을 찾아서 외곽선을 그리시오.
- 외곽선의 길이는 외곽선을 구성하는 점의 개수로 정의한다.

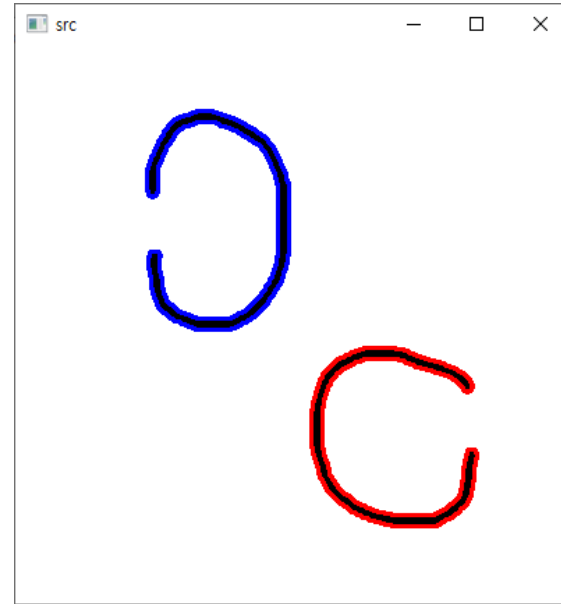
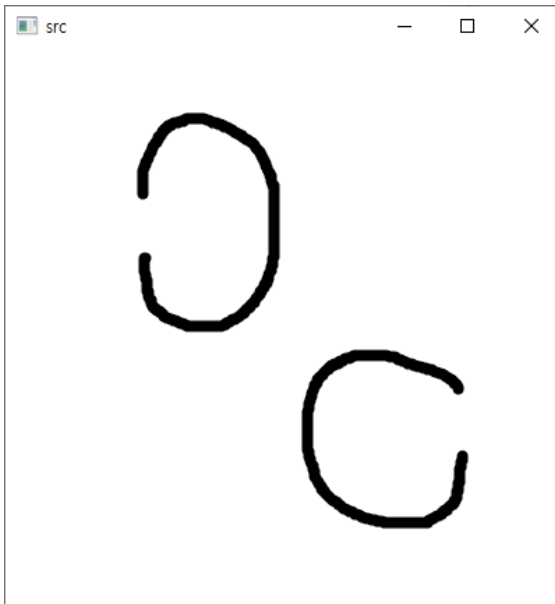
```

D:\Users\W2sungryul\Dropbox\Lecture\2022년1학기\컴퓨터...
0번째 외곽선 길이:268
1번째 외곽선 길이:328
2번째 외곽선 길이:354
3번째 외곽선 길이:98
길이가 최대인 외곽선:2번째, 길이는 354
  
```



# 실습과제5

- char\_c.png 영상에서 열려있는 방향을 구하시오. 열린방향이 왼쪽이면 외곽선을 파란색으로 그려주고 열린방향이 오른쪽이면 외곽선을 빨간색으로 그리시오.



D:\Users\2sungryul\Dropbox\Lecture\2022년1학기\컴...  
0번째 문자(빨간색):오른쪽으로 열림  
1번째 문자(파란색):왼쪽으로 열림

# 과제 제출 방법

- 소스파일, 라인단위 코드설명, 실행결과를 하나의 PDF 파일로 작성하여 과제게시판에 제출하시오. 이외 양식은 0점 처리함
- 제출기한은 과제 게시판을 참고할 것.